

Jayhawk Engineering Design Laboratory
3005 Eaton Hall, Lawrence, Kansas 66045

September 28th, 2025

FROM: Leo Cabezas, Daniel Gonzales, Kieran Egan, Fahad Almarri (Team #1)

TO: JEDL Director of Engineering

SUBJECT: Project #1 – Short-range IR Telecommunications System

1. Statement of Purpose

This project aims to determine the feasibility of a new Jayhawk Engineering Company (JECO) product, an infrared (IR) telecommunications system, by constructing a functional prototype. The goal is to simultaneously maximize the effective transmission distance between the transmitter and receiver, while mitigating errors in the signal. The transmitter (TX) system must be able to transmit binary data with an Arduino Nano over infrared light. The receiver (RX) system must be able to recover the IR-transmitted data and apply error correction.

Product design limitations:

- The transmitter and receiver circuits must include an Arduino Nano.
- The transmitter and receiver circuits are to be implemented on circuit boards that are produced on the shop laser circuit board maker or soldered on a Vector board.
- Power is to be provided by laboratory power supplies or batteries.
- The transmitter and receiver should be independently powered.
- All components must come from the KU EECS shop.
- The transmitter and receiver are to be constructed on separate circuit boards.

Design objectives given by the Director of Engineering:

- Design and implement a transmitter (TX) circuit that converts Arduino-generated digital signals into optical signals using infrared LEDs.
- Design and implement a receiver (RX) circuit that detects the transmitted optical signals and converts them back into electrical signals for Arduino processing.
- Apply current limiting resistors and transistors driver stage to ensure safe and efficient operation from LEDs without damaging the components.
- Prototype, test and debug the circuits using variable resistors, probing techniques, and rapid prototyping tools.
- Verify the complete system by ensuring that the transmitter successfully sends data and the receiver interprets it accurately.

2. Theory and Design

2.1. Circuit Theory

For the transmitter circuit, the components we used were an Arduino Nano, resistors, QED234 LED diodes, and a 2N2222A NPN transistor. The variable resistor had a tunable knob and was straight forward to implement characterized by the equation (Ohm's law).

$$V = IR \quad (1)$$

For the two QED234 LED diodes, we referred to the datasheet [1] for the continuous forward current, 100 mA, and the forward voltage, 1.6 V. For the 2N2222A NPN transistor we used equations assuming the transistor is in saturation to find resistor values.

$$R_1 = \frac{V_{DD} - (V_f + 0.2)}{i_c} \quad (2)$$

$$R_2 < \beta \left(\frac{v_{max} - 0.7}{i_c} \right) \quad (3)$$

The Arduino supplies 5V across a resistor for our base current of the NPN transistor. When base current is present, this allows current to flow through the LEDs which makes them emit IR. The collector side is powered with 9V DC from the bench power supply. We used a variable resistor on the collector side to vary the current going through the LEDs, so that we can adjust for the range at which the LEDs emit infrared light. The 5V signal from the Arduino can be pulsed, allowing our LEDs to emit with each pulse, this is what allows us to send data.

2.1.b. Receiver

For the receiver circuit, the components we used were an Arduino Nano, resistors, a QSD123 phototransistor, and an LM318 op amp. Resistors were once again easy to implement with the simple Ohm's law equation. For the QSD123 phototransistor we referred to the datasheet [2] for specific values, like max power dissipation of 100 mW, and how the device was to be used. In our circuit we used the phototransistor like an NPN transistor with the base current coming from emitted IR. For the LM318 op amp, we used it in the comparator mode characterized by the following equation.

$$V_{out} = A(V_+ - V_-) \quad (4)$$

In our circuit, we used the Arduino 5V DC output to power the collector of the phototransistor along with a resistor in series for current flow. At the output/emitter of the phototransistor we used a voltage divider to feed in a reasonable voltage to the op amp positive input. To power the op amp, we biased it with +9V on the positive supply and -9V on the negative supply from the bench power supply. To supply the negative op amp input, we tapped the positive power supply voltage with a voltage divider into the negative input. At the output of the op amp, we again used a voltage divider to bring it down to a reasonable voltage for the Arduino. We

included a variable resistor at the op amp output voltage divider to dial in correct voltage values for reliable communication at a set distance.

2.2. Circuit Design

2.2.a. Transmitter

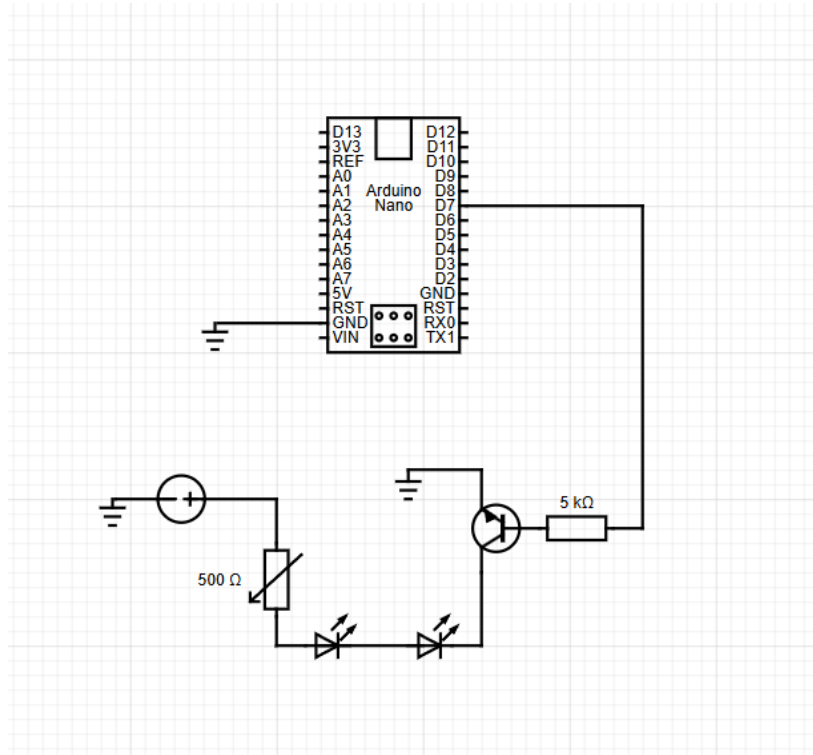


Figure 1 – Electronic Circuit Diagram Transmitter

The transmitter circuit is designed around the Arduino Nano, which generates the control signal to drive and infrared (IR) LED pair. The LED is responsible for converting the digital signal from the Arduino into an optical signal that can be transmitted through free space to the receiver. To control the LED current and ensure safe operation, variable resistor (500Ω) is placed in series with the LEDs (5). This allows fine adjustments of current to avoid exceeding the LED's maximum current and allows us to be able to modify it for each distance, during testing, this flexibility was crucial, since it ensured reliable operation while preventing damage to the components.

$$I_{LED} = \frac{V_{CC} - V_{LED}}{R_{var}} \quad (5)$$

A transistor is used as a switching element, providing current amplification. The Arduino output signal is applied to the transistor's base through a ($5k\Omega$) resistor, which limits the base current and ensures the transistor operates in saturation mode when supplied voltage (6). This setup allows the Arduino to control the LED pair without directly sourcing the full current required, protecting the pins.

$$I_B = \frac{V_{OH} - V_{BE}}{R_B} \quad (6)$$

2.2.b. Receiver

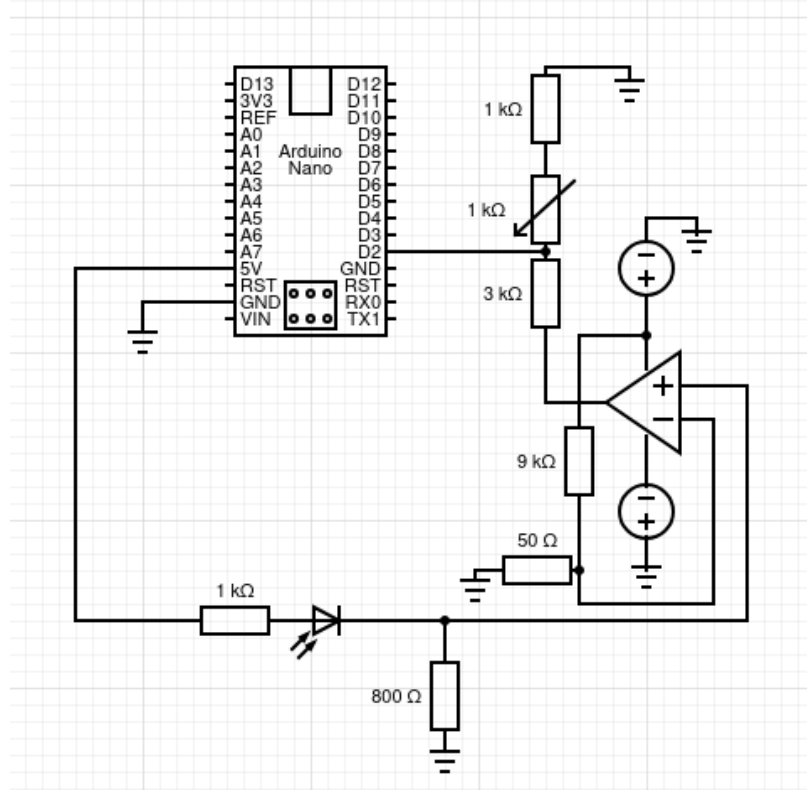


Figure 2 – Electronic Circuit Diagram Receiver

The receiver circuit is designed to detect the transmitted infrared (IR) signal, amplify it, and provide a clean digital signal to the Arduino Nano for processing. The main sensing element is a photodiode, which converts the incoming IR light into a small current. This current is then converted to a voltage through a (1kΩ) resistor (7), producing a signal proportional to the received light intensity. To improve sensitivity, the photodiode is reversed-biased, which enhances its response speed linearity.

$$V_{in} = I_{PD} \cdot R \quad (7)$$

Since the photodiode output is weak, the signal is directed into an operational amplifier (op-amp) stage. The op-amp is configured as a non-inverting amplifier, where the gain is determined by the resistor network (1kΩ, 1kΩ variable resistor and 3kΩ). The inclusion of the variable resistor allows adjustment of the gain during testing. See (8).

$$V_{out} = \left(1 + \frac{R_f}{R_{in}} \right) V_{in} \quad (8)$$

The amplified signal will be then routed to the Arduino, which interprets it as the received IR signal. During testing, we observed that ambient sunlight caused interference with the photodiode, resulting in false triggers. This limitation was noted and followed by careful adjustment of the variable resistor and by testing controlled lighting conditions.

2.3. Data Communication Theory

2.3.a. Data encoding scheme

Regarding Pulse Frequency Modulation and bit encoding

Pulse-Frequency Modulation (PFM) is a method of digital data on a pulse train by varying the frequency at which pulses are transmitted, while keeping their width and amplitude constant. While performing preliminary tests on our circuit design to determine its feasibility, it was found that pulse width may be severely affected by the distance between transmitter and receiver. It is mainly for this reason that our team decided to pursue PFM over other encoding schemes. The following equation succinctly describes the mathematical basis of PFM:

$$f = f_{\text{car}} + B \cdot f_{\text{dev}} \quad (9)$$

Here, f is the instantaneous frequency of a pulse in the pulse train, f_{car} is the carrier frequency of the pulse train, f_{dev} is the deviation frequency, and B is the ‘signed’ equivalent of the transmitted bit ($B = 1$ if the transmitted bit is **1**; $B = -1$ if the transmitted bit is **0**). Taking the reciprocal of (9), we obtain

$$T = \frac{1}{f_{\text{car}} + B \cdot f_{\text{dev}}} \quad (10)$$

which is the instantaneous period of a given pulse in the pulse train. Therefore, to transmit a bit, a pulse of arbitrary width (but less than $1 / f_{\text{car}}$) is sent, and then a time delay T is introduced before sending the next pulse in the train. This time gap between pulses encodes the binary data (see Figure 8 for an example).

Regarding data frame and message structures

For the purposes of this proof-of-concept project, only ASCII characters are PFM-encoded and transmitted by our system. In our encoding scheme, each 8-bit ASCII character is packaged in a 16-bit data ‘frame’, whose structure is

Start bit	Nibble 1	Parity 1	Nibble 2	Parity 2	End bit
S	$d_1d_2d_3d_4$	$p_1p_2p_3$	$d_5d_6d_7d_8$	$p_4p_5p_6$	E

for an ASCII character with bits $d_1d_2d_3d_4d_5d_6d_7d_8$, where the parity bits for each nibble are generated by a slightly modified version of the Hamming(7,4) error correction algorithm. See Figure # for an example of a data frame. Moreover, our encoding scheme defines a ‘message’ as a sequence of frames encoding a string of characters. Each message must always begin and end with agreed-upon start and end frames, respectively, (see Section 3.2), so its boundaries are well-defined for the receiver. Message length is theoretically unlimited with PFM encoding.

2.3.b. Data decoding scheme

Regarding PFM bit decoding

To decode data bits transmitted following the PFM-encoding scheme from Section 2.3.a., the receiver must first determine the instantaneous period T_{meas} between two consecutive pulses. Solving for B in (10), we obtain

$$B_{\text{meas}} = \frac{1}{f_{\text{dev}}} \left(\frac{1}{T_{\text{meas}}} - f_{\text{car}} \right) \quad (11)$$

which is the ‘signed’ bit measured by the receiver. In practice, B_{meas} will rarely be exactly 1 or -1 . To recover the data bit, we only consider the sign of B_{meas} , so

$$b_{\text{data}} = \begin{cases} 1 & \text{if } B_{\text{meas}} > 0 \\ 0 & \text{if } B_{\text{meas}} < 0 \end{cases} \quad (12)$$

where b_{data} is the data bit recovered from the instantaneous period between two consecutive pulses. See Section 3.2 for the corresponding implementation details.

Regarding data frame and message decoding

To decode a full message, the receiver system continuously monitors a 16-bit wide input buffer containing decoded data bits. When the buffer matches the predefined message start frame, message decoding begins. Decoding continues until the buffer matches the message end frame, at which point decoding stops and the message is considered complete.

During message decoding, the receiver monitors the buffer for valid data frames. When the start and end bits are detected, the buffer's contents are saved for further processing to extract its data and parity bits. After the frame is saved, the buffer is immediately flushed to prepare for the next incoming frame.

2.2.c. Error correction scheme

The parity bits described in Section 2.3.a are obtained using a modified version of the Hamming(7,4) error detection and correction algorithm. In short, our version only changes the position of the 3 parity bits with respect to the data bits. Normally, Hamming(7,4) uses a $p_1p_2d_1p_3d_2d_3d_4$ bit layout. However, for the sake of simplicity and for debugging purposes, we decided on a $d_1d_2d_3d_4p_1p_2p_3$ bit layout. This modifies how the Hamming syndrome is calculated (see the GitHub repository in Section 3.2), but it does not affect the algorithm’s functionality.

3. Implementation

3.1. Circuit Implementation

3.1.a. Transmitter implementation

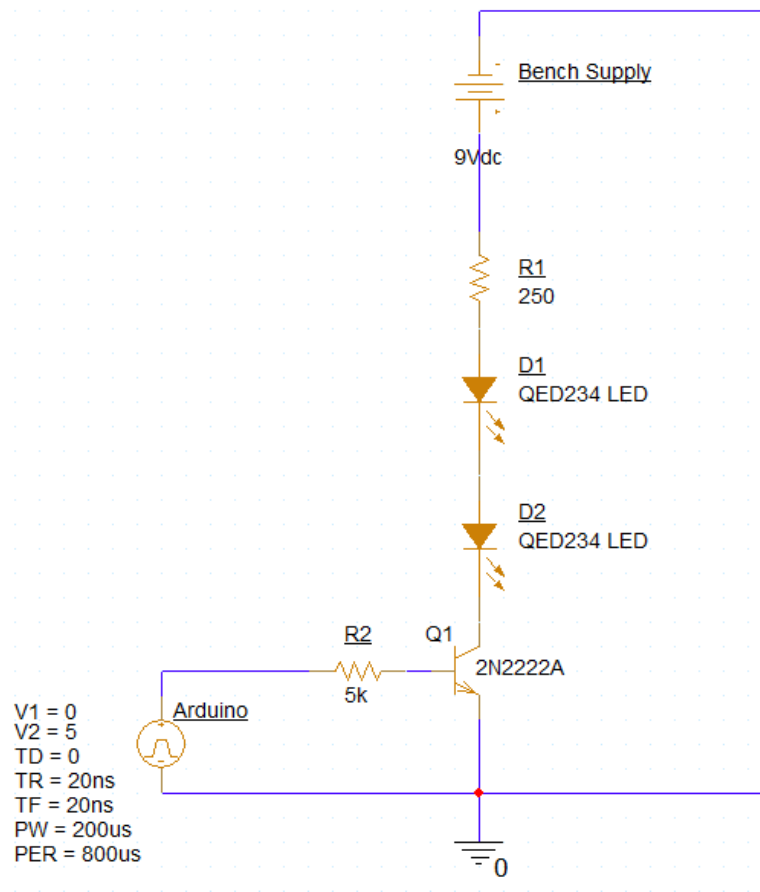


Figure 3 – PSpice drawing of TX

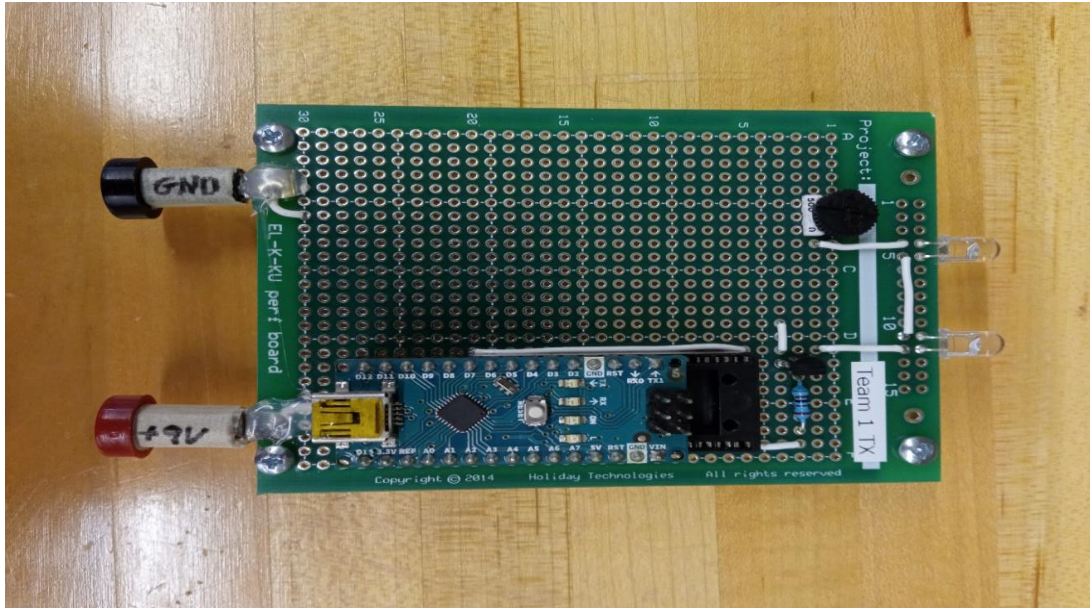


Figure 4 - Final TX Circuit

Placement of the components for the transmitter on the Vector board was straight forward. The placement of the PSpice components mapped easily and was similar to our electronic circuit diagram. The only difficult components to implement were the 40-pin socket connector for the Arduino, and the standard banana cable ports for the bench power supply. For the 40-pin socket, we removed all pins except those necessary for the circuit functionality to allow it to more easily fit onto the board. For the banana cable ports, we used a wire soldered to the connector and a plane on the board. Our ground and +9V from the bench supply were connected to separate planes on the Vector board.

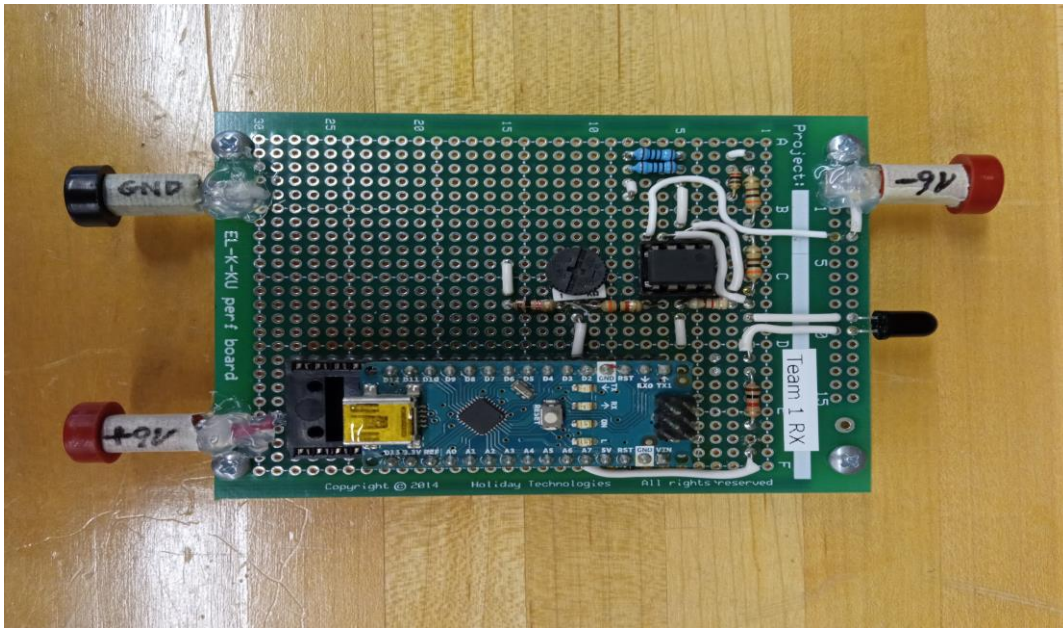


Figure 5 - Final RX Circuit

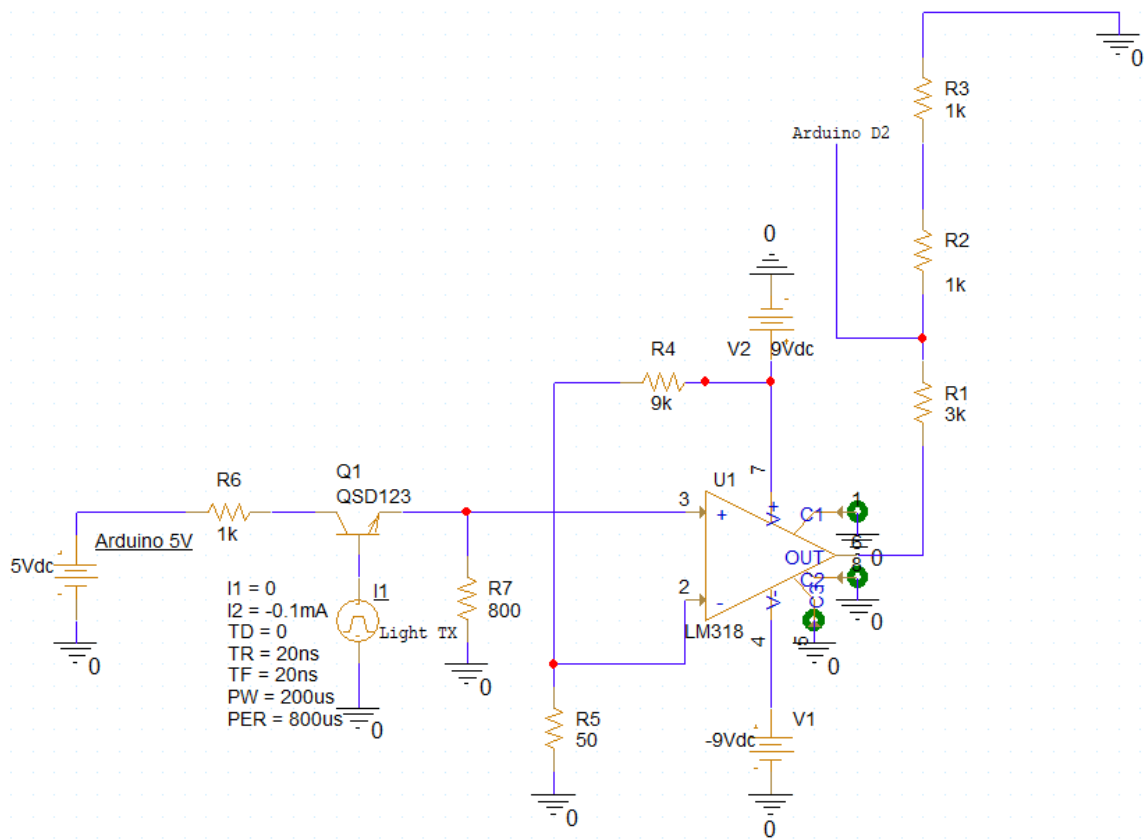


Figure 6 – PSpice RX Circuit Drawing

3.1.b. Receiver implementation

Placement of the components was again straight forward for the receiver. Even though more complex, it mapped well onto the Vector board from the PSpice drawing and the electronic circuit diagram. We again had the same problem with the 40-pin socket where we removed all pins except those necessary for the circuit functionality to allow it to more easily fit onto the board. For the standard banana cable connectors, we again used the same technique for the ground and +9V, where we soldered a cable from the connector to a plane on the board. The challenge came when connecting the -9V to bias the op amp, as the board only had two planes, we couldn't connect it to a ground plane. Instead, we created a direct path for the -9V connection, soldering a specific path for the banana cable connector. Other than those issues, we had the general issue of routing connections between small components that became intricate.

Overall, the Vector board implementation went well for both the transmitter and receiver. Our designs translated very well from PSpice drawings, electronic circuit diagrams, and prototype circuits on breadboards. All parts tolerances were accounted for and there were no parts substitutions from the planned parts we originally used.

3.2. Data Communication Implementation

Please find the Arduino C++ source code for our communication system in the following GitHub repository: https://github.com/leo-cabezas-amigo/Arduino_IR_Telecom_System.

3.2.a. Data encoding implementation

Regarding the bit encoding implementation

To implement data encoding (as described in Section 2.3) in the Arduino Nano platform, we made use of a slightly modified version of equation (10), given by

$$T_{us} = \left\lceil \frac{10^6}{f_{car} + B \cdot f_{dev}} \right\rceil \quad (13)$$

where T_{us} is the time delay (in microseconds) before sending the next pulse. In `tx_nano.ino`, this calculation is performed directly by the `instantPeriod_us()` function.

Regarding the message and frame encoding implementation

In our implementation, the message to be sent (e.g. `'hello_world'`) is hard-coded in the `tx_nano.ino` source code. Before transmission commences, the `encodeMessage_us()` function is called in `setup()`, translating the message string into an array of vectors. Each entry of the array corresponds to a single data frame, which is encoded as a vector of instantaneous periods T_{us} (in microseconds). Data frames are obtained by calling the `packetToFrame()` and `encodeFrame_us()` functions, which translate an 8-bit ASCII character into a vector of corresponding T_{us} .

Note that manual error injection may be performed directly in *encodeMessage_us()* via the *injectError()* function. In the ‘TX PARAMETERS’ section of the code, the *target_frame_idx* and *target_bit_idx* arrays identify which frames will be affected (by index, starting from 0) by the error injection, and which exact bit within that frame will be affected, respectively. Two copies of these arrays are specified, each targeting one of the two nibbles within a frame.

3.2.b. Data decoding implementation

Regarding data bit decoding

To measure the period between two consecutive voltage pulses, we made use of the Arduino Nano’s *attachInterrupt()* function, with the ‘RISING’ parameter. When the Nano detects a rising edge in the corresponding digital pin, it calls the *risingEdge()* handler function, which logs the time stamp and calculates the instantaneous period given the last pulse time. The *getBitFromPeriod_us()* function translates this instantaneous period into its corresponding data bit, as outlined in Section 2.3.b.

Regarding the message decoding implementation

To decode an entire message, the Arduino Nano closely follows the indications given in Section 2.3.b. The Nano monitors a 16-bit data bit buffer and identifies data frames with the *frameDetect()* function. If a frame is detected, the received message is updated using the *updateMessageBuffer()* function, which implicitly calls *getHammingCorrectedNibble()* to enact error correction on the data nibbles as needed. Statistical data is gathered for every received message (the minimum, maximum, and average data reception frequencies). An error percentage (bits in error out of total number of bits) is also calculated. Once the message end frame is detected, the received message and its statistical data are printed to the serial monitor.

4. Laboratory Evaluation

4.1. Configuration

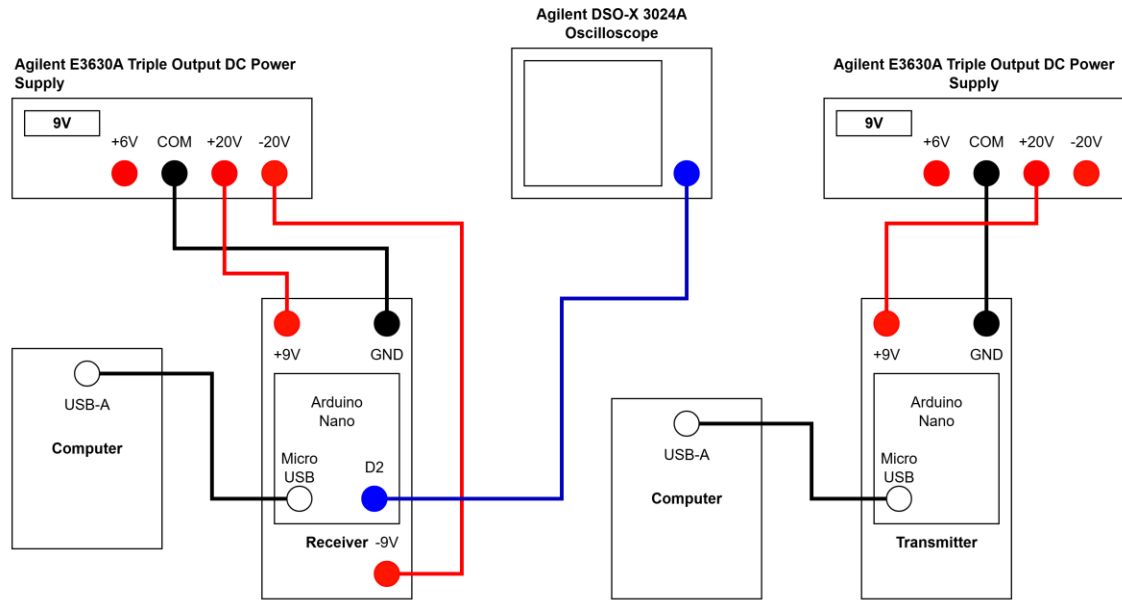


Figure 7 – Test Equipment Setup for Evaluation

4.2. Procedure

We began by assembling both the transmitter (TX) and receiver (RX) circuits on breadboards to allow for quick testing and modifications. The (RX) circuit was built and tested first using a standard TV remote controller as the infrared source to verify that photodiode and amplifier stages were working correctly. After confirming basic reception, we proceeded to build the (TX) circuit.

Once both circuits were complete, we applied power to the system and checked the LED current using a multimeter to ensure safe operating conditions. The photodiode output and amplifier stage were then probed with the oscilloscope to verify that the received signals matched the transmitted waveform. At this stage, we used the function generator to supply test signals to the transmitter, while the regulated power supply provided consistent bias voltage.

We had issues that encountered us at the beginning. The receiver (RX) generally responded to the TV remote controller, but performance was inconsistent with environmental light, which interfered with the photodiode. In addition, the oscilloscope displayed unstable signals when current is not correctly adjusted. Several adjustments were made during the testing. A variable resistor was used to tune the biasing and improve the clarity of the signal received.

Finally, after debugging and calibration, we measured the effective transmission range. The circuit successfully transmitted data across a distance of 3 meters without error, meeting and exceeding the expected performance requirements.

5. Evaluation of Test Data

5.1. Data

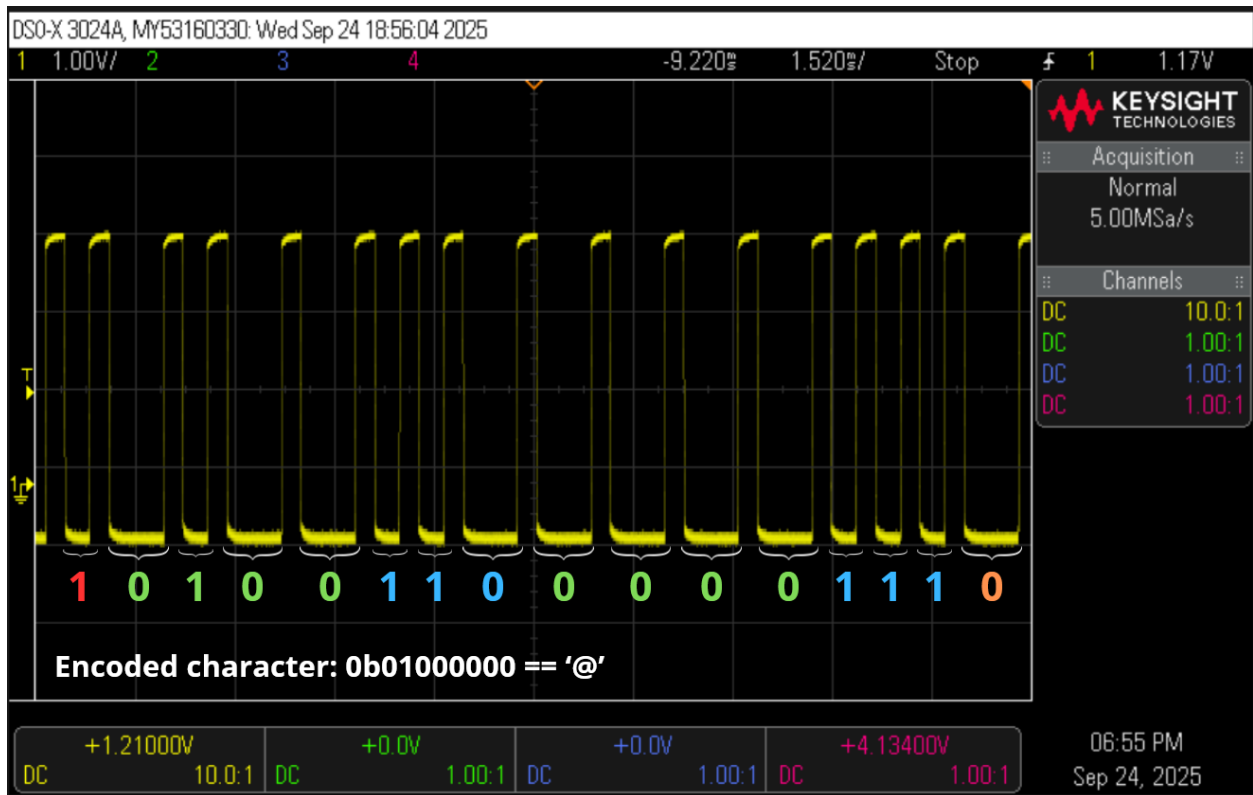


Figure 8 – Example of a data frame received by the RX system, originally transmitted at 1200Hz with a frequency deviation of 300Hz (highly exaggerated for visualization purposes). The red and orange bits are the start and end bits. The green bits are the data bits. The blue bits are the parity bits corresponding to each of the data nibbles. This encodes a '@' character.

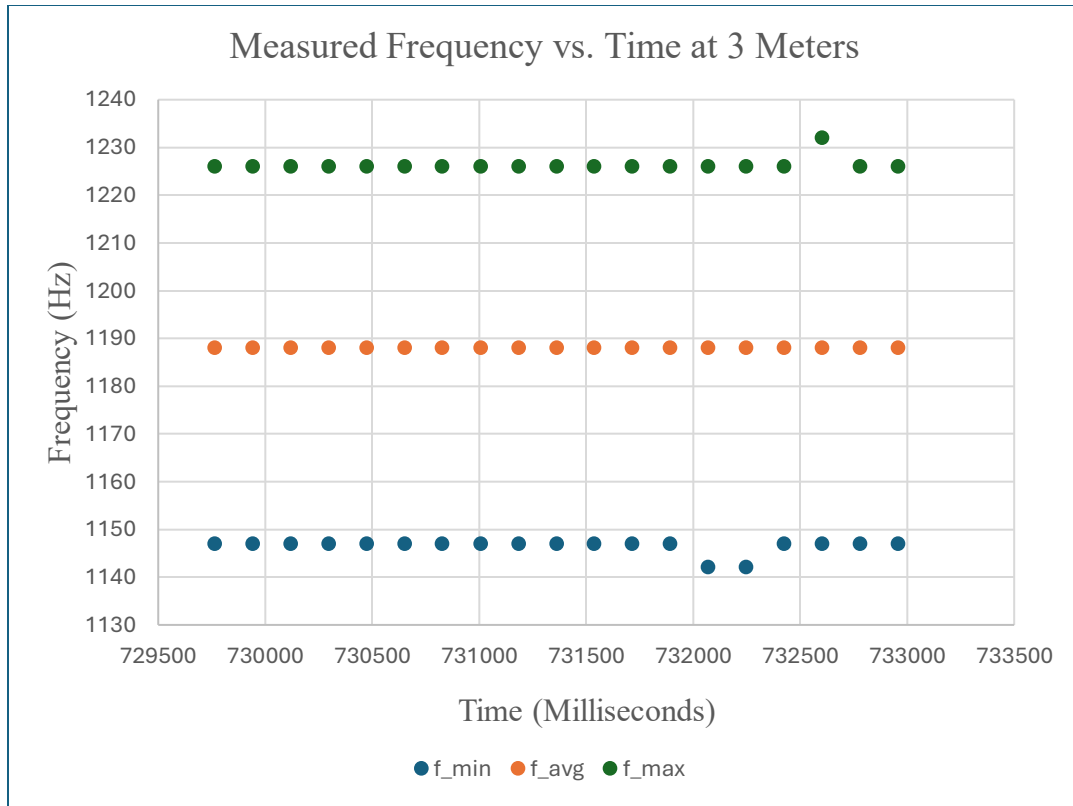


Figure 9 - Measured Frequency vs. Time

TABLE 1 – SIGNAL FREQUENCY ACROSS DIFFERENT RANGES AND TESTS

Demo	Mean f (kHz)	Min f (kHz)	Max f (kHz)	Std Dev (Hz)	%Dev (from 1.2kHz)
30 Hz (1m)	1.1906	1.15	1.23	27.9	2.33%
50 Hz (1m)	1.1856	1.16	1.25	50.7	4.22%
30 Hz (3m)	1.1861	1.15	1.22	27.1	2.26%
50 Hz (3m)	1.1836	1.12	1.25	50.1	4.18%

Output	Serial Monitor	X
Message (Enter to send message to 'Arduino Nano' on 'COM6')		
[t: 2029865]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "w}}Q_the_best_language_ever"]
[t: 2030259]	[f_min: 1147; f_avg: 1188; f_max: 1232]	[err%: 3.7%] [msg: "w}}Q_the_best_language_ever"]
[t: 2030652]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "w}}Q_the_best_language_ever"]
[t: 2031045]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "w}}Q_the_best_language_ever"]
[t: 2031438]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "w}}Q_the_best_language_ever"]
[t: 2031832]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "w}}Q_the_best_language_ever"]
[t: 2032225]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "w}}Q_the_best_language_ever"]
[t: 2032618]	[f_min: 1147; f_avg: 1188; f_max: 1232]	[err%: 3.7%] [msg: "w}}Q_the_best_language_ever"]
[t: 2033011]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "w}}Q_the_best_language_ever"]
[t: 2033404]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "w}}Q_the_best_language_ever"]
[t: 2033798]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "w}}Q_the_best_language_ever"]
[t: 2034191]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "w}}Q_the_best_language_ever"]
[t: 2034585]	[f_min: 1142; f_avg: 1188; f_max: 1232]	[err%: 3.7%] [msg: "w}}Q_the_best_language_ever"]
[t: 2034978]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "w}}Q_the_best_language_ever"]
[t: 2035372]	[f_min: 1142; f_avg: 1188; f_max: 1232]	[err%: 3.7%] [msg: "w}}Q_the_best_language_ever"]
[t: 2035765]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "w}}Q_the_best_language_ever"]
[t: 2036158]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "w}}Q_the_best_language_ever"]
[t: 2036551]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "w}}Q_the_best_language_ever"]
[t: 2036944]	[f_min: 1142; f_avg: 1188; f_max: 1232]	[err%: 3.7%] [msg: "w}}Q_the_best_language_ever"]

Figure 10 -8 Transmission with Known Error and No Error

Output	Serial Monitor	X
Message (Enter to send message to 'Arduino Nano' on 'COM6')		
[t: 25594]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "c_is_the_best_language_ever"]
[t: 25988]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "c_is_the_best_language_ever"]
[t: 26381]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "c_is_the_best_language_ever"]
[t: 26774]	[f_min: 1142; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "c_is_the_best_language_ever"]
[t: 27167]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "c_is_the_best_language_ever"]
[t: 27560]	[f_min: 1147; f_avg: 1188; f_max: 1232]	[err%: 3.7%] [msg: "c_is_the_best_language_ever"]
[t: 27954]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "c_is_the_best_language_ever"]
[t: 28347]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "c_is_the_best_language_ever"]
[t: 28740]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "c_is_the_best_language_ever"]
[t: 29133]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "c_is_the_best_language_ever"]
[t: 29527]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "c_is_the_best_language_ever"]
[t: 29920]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "c_is_the_best_language_ever"]
[t: 30313]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "c_is_the_best_language_ever"]
[t: 30706]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "c_is_the_best_language_ever"]
[t: 31099]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "c_is_the_best_language_ever"]
[t: 31493]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "c_is_the_best_language_ever"]
[t: 31886]	[f_min: 1142; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "c_is_the_best_language_ever"]
[t: 32279]	[f_min: 1142; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "c_is_the_best_language_ever"]
[t: 32672]	[f_min: 1147; f_avg: 1188; f_max: 1226]	[err%: 3.7%] [msg: "c_is_the_best_language_ever"]

Figure 11 - Transmission with Known Error and Error Correction

5.2. Interpretation

Through testing the maximum distance of our IR communication system, we discovered that the transmission became overwhelmed with errors exceeding 3 meters (~9.8 ft) in length. During this test, the frequency deviation remained steady at ± 50 [Hz] away from 1200 [Hz] and 50 [mA] of current. Additionally, we tested our system's lowest possible deviation from the prescribed frequency of 1200 [Hz] at our maximum distance. We found that we could get a minimal deviation of 30 [Hz] or $\pm 2.5\%$ of 1200 [Hz]. This is well within the $\pm 20\%$ allowance given by

the Director of Engineering. The research for this project uncovered a wide range of maximum theoretical and practical lossless distances for a two LED IR communication system. A typical TV remote, for example, will lose effectiveness beyond 5-10 meters. Given these facts and the additional design limitations of the project, a range of 3 meters is a promising result.

In addition to correctly receiving the raw data transmission, we tested our error correction algorithm, described above, to determine if it was effective at detecting and correcting bit-flips. Figure 10 demonstrates how the serial output looks when a transmission has errors. As seen next to err%, the receiver can detect up to 2 bits of error per 4 bits of information. These detections form the basis of the error percentage. This value represents the percentage number of bits that are flipped according to the detection algorithm. As presented in figure 11, the receiver is able to also correct bit-flips, assuming that there is, at most, one bit flip per 4 information bits. Thus, the receiver is able to successfully decode and recover the transmitted string.

5.3. Conclusions

Given the project objective of creating an IR telecommunications system using a 1200 [Hz] frequency, the final product was incredibly effective. As demonstrated in the test results, our communication system could transmit information up to 3 meters in length with no errors. Furthermore, our error correction algorithm could detect two bits of error and correct for a single bit-flip for every 4 bits of information transmitted. Our solution was robust, and care was taken when designing and assembling the product. Overall, our project was successful, and we met and exceeded the required criteria.

6. References

- [1] onsemi, "QED234 Plastic Infrared Light Emitting Diode," Data Sheet QED234/D. [Online]. Available: <https://www.onsemi.com/pdf/datasheet/qed234-d.pdf>
- [2] onsemi, "QSD123, QSD124 Plastic Silicon Infrared Phototransistor," Data Sheet QSD123/D, Rev. 3, August 2023. [Online]. Available: <https://www.onsemi.com/pdf/datasheet/qsd123-d.pdf>
- [3] University of New Brunswick (UNB), "Error Correction and the Hamming Code". Author and date unknown. Available online at: <https://www.ece.unb.ca/tervo/ee4253/hamming.shtml>